Deep Learning

Francisco J. Palmero Moya

September 3, 2023

Abstract

Deep Learning for image classification has revolutionized the field of computer vision by leveraging neural networks to automatically extract and learn intricate features from raw image data. This approach, characterized by its depth and complexity, enables the automated categorization of images into predefined classes or labels, yielding remarkable accuracy in tasks such as object recognition, scene understanding, and pattern detection. By utilizing convolutional neural networks, deep learning models have surpassed traditional machine learning methods offering unparalleled performance in a wide range of applications. This document presents a study of different neural networks models varying in complexity for a typical image classification problem. The applied models are discussed in terms of their features, training and test performance, topology, and complexity.

1 Introduction

In palynology, the visual classification of pollen grains from different species has long been a challenging task, traditionally reliant on expert human examination through microscopes. Complete automation of this process holds the potential to significantly conserve resources, particularly the laborious hours of doctoral students, and promises substantial advancements, not only in allergy-related information systems but also in various fields such as paleoclimatic reconstruction, quality control of honey-based products, forensic evidence identification in criminal investigations, and tissue dating and tracing. This document addresses the pressing issue of identifying pollen grains from two distinct species originating in New Zealand. Both species produce pollen grains that are deemed virtually indistinguishable, characterized as monads, isopolar, angulaperturate, syncolpate, tectate, and triangular in polar view. While there exists a slight disparity in their average size, there is significant overlap in the size distributions of both species, as well as subtle variations in grain shape when photographed in equatorial view, posing considerable challenges for human differentiation.

To address this classification problem, we have employed a diverse set of models varying in complexity. Our approach begins with a baseline logistic regression model, a classic choice for binary classification tasks, which serves as a fundamental benchmark. Additionally, we have implemented a neural network architecture featuring multiple hidden layers, which enhances our classification capabilities by extracting meaningful features from the data. Furthermore, to leverage the spatial information and structural characteristics of the grains, we applied a convolutional neural network (CNN), a specialized deep learning architecture adept at image analysis tasks. This comprehensive methodology aims to explore the effectiveness of these three distinct approaches in achieving accurate and automated classification of pollen grains from the specified New Zealand species.

2 Methods

2.1 Logistic Regression

Logistic regression serves as our initial approach in this study. This model is a widely-used tool for binary classification tasks. It operates by modeling the probability of one of the two possible outcomes based on a set of input features, employing a logistic function to ensure predictions fall within the interval $I = [0,1]$. Despite its simplicity, logistic regression provides valuable insights into the problem at hand, allowing us to establish a baseline for performance assessment.

Logistic regression can be viewed as a special case of the simplest neural network. While neural networks typically consist of multiple interconnected layers of neurons, logistic regression represents the most elementary form of this architecture. In a neural network, a logistic regression unit corresponds to a single neuron with no hidden layers. The input features are linearly combined, and the resulting weighted sum is passed through an activation function, typically the sigmoid function, to produce a probability score for binary classification.

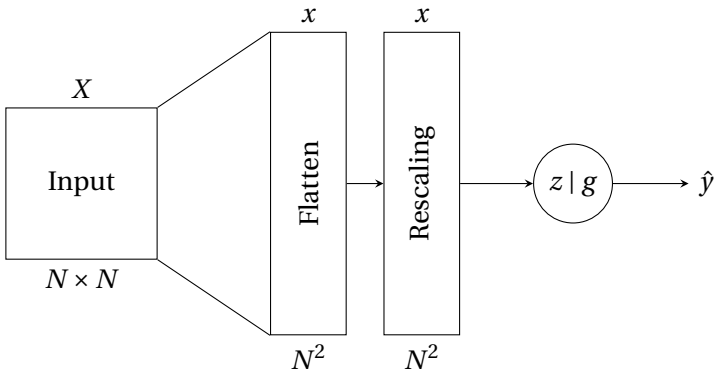

Figure 1: Logistic regression as the simplest neural network. The input image is flattened and then passed through a rescaling layer. Output layer takes a set of input features X, applies weights to those features, calculates the weighted sum Z, and then passes it through an activation function g. The model output is $\hat{y}$.

Figure 1 shows the logistic regression as the simplest neural network structure applied in our problem. The input image is flattened and then passed through a rescaling layer. The rescaling consist on a normalization of pixel values between 0 and 1. Since the image is originally an 8 bits image, the normalization implies divide by 255 each pixel value. Finally, the output layer takes a set of input features X, applies weights to those features, calculates the weighted sum $z = Wx + b$, and then passes it through an activation function g. The output $\hat{y}$ is then classified as in either one class or the other if $g(z)$ is greater or lower than a given threshold for accuracy measure.

As we progress in our analysis, we will contrast the outcomes of logistic regression with those of more sophisticated neural network models to assess the potential benefits of deep learning in tackling the task of pollen grain classification.

Parameters configuration

The output layer corresponds to a single neuron where the activation is chosen to be the sigmoid function

$$\sigma(z) = \frac{1}{1+e^{-z}} \in [0,1] \tag{1}$$

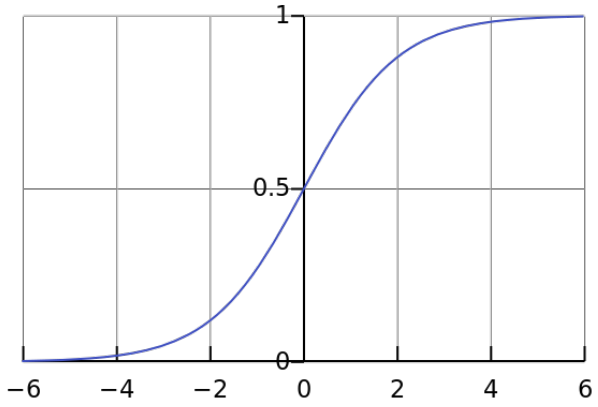

Figure 2: Sigmoid function σ

The weights W are randomly initialized using Glorot Uniform method [1]. On the other hand, the bias is initialized at zero. There is no kernel or bias regularization in our implementation to kept it simple.

The loss function is the typical selection for binary classification, namely the binary cross-entropy

$$L(y,\hat{y}) = -y\log\hat{y} - (1-y)\log\big(1-\hat{y}\big) \tag{2}$$

where y represents the ground truth and is either 0 or 1.

2.2 Multilayer Perceptron

Our second model in this study is a neural network featuring two hidden layers, also known as multilayer perceptron.

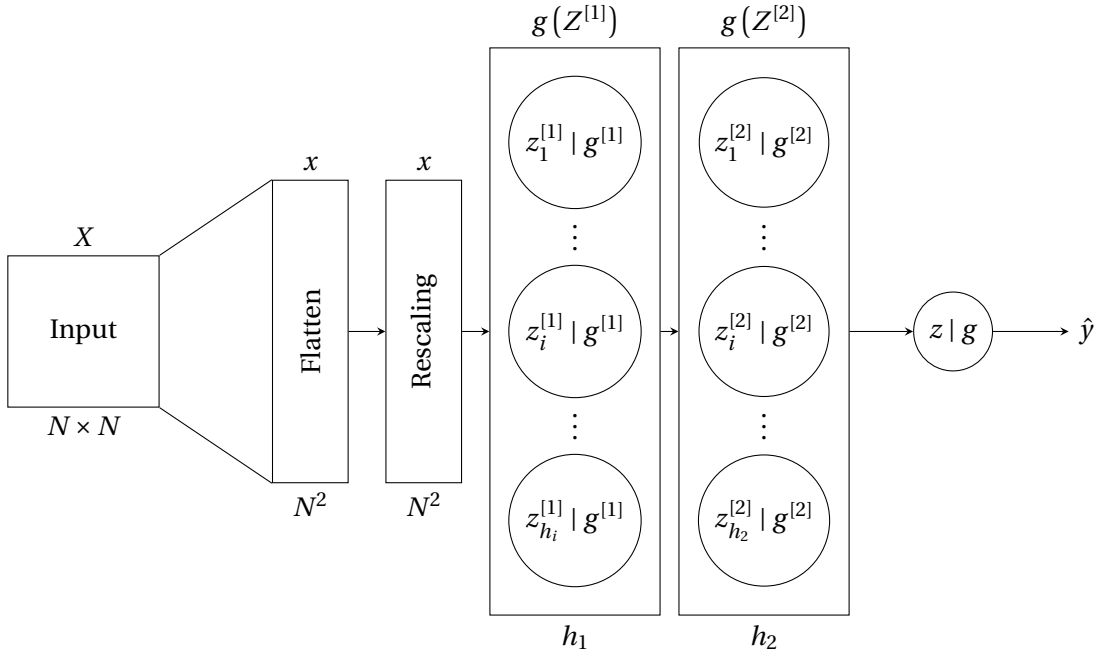

Figure 3: Multilayer perceptron of two fully connected hidden layers. The arrows $\longrightarrow$ points to the next layer, instead of to any specific unit to simplify our notation. The superscript in $g^{[i]}$ emphasize that activation function at each layer may be different. $Z^{[i]}$ is the matrix representation of vectors $z_j^{[i]}$ for each $j = 1,...,h_i$.

Figure 3 shows the multilayer perceptron as the second neural network structure applied in our problem. The core structure is similar to the logistic regression in Figure 1, the main difference is the two hidden layers added to the model. These hidden layers are fully connected, therefore the weights $W^{[i]}$ and bias $b^{[i]}$ have the following dimensions[1]

$$\begin{cases} \dim W^{[1]} &= N^2 \times h_1 \\ \dim W^{[2]} &= h_1 \times h_2 \\ \dim W &= h_2 \end{cases} \qquad \begin{cases} \dim b^{[1]} &= h_1 \\ \dim b^{[2]} &= h_2 \\ \dim b &= h_2 \end{cases}$$

Parameters configuration

The output layer corresponds to a single neuron with sigmoid activation (1) analogous to the logistic regression. The activation of units belonging to both hidden layers are the rectified linear unit (ReLU) functions

$$g^{[i]}(z) = \frac{z+|z|}{2} = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}, \ \forall i = 1,2 \tag{3}$$

Weights and bias initialization, and loss functions are identical to the logistic regression. The hidden layers have $h_i = 128$ units both of them.

[1] The bias vectors have different dimensions to match dimensions with the matrix multiplication, however, the number of parameters is the showed in the equations.

2.3 Convolutional Neural Network

Our final and most sophisticated model in this endeavor is the Convolutional Neural Network (CNN). Unlike the previous models, the CNN is tailored specifically for image-related tasks, making it exceptionally well-suited for the intricate nature of our pollen grain classification problem. By implementing convolutional layers, pooling layers for spatial down-sampling, and fully connected layers for decision-making, our CNN embodies a comprehensive framework for image classification.

Figure 4 shows the CNN structure used in our problem. The core structure of the multilayer perceptron appears in the CNN as a last stage. Indeed, before the image is flattened it pass through two convolutional layers consisting on: a convolutional operator of f_i filters, $k_i \times k_i$ kernel size, and activation function $\phi^{[i]}$, a Max-pooling layer of size $m_i \times m_i$ and a drop-out operator of rate d_i.

The convolutional blocks consist on: a convolutional layer, a max-pooling operation and a drop-out application. In a convolutional layer, we perform a convolution operation between the input data and a set of learnable filters (also called kernels). This operation helps the network learn features from the input data. After a convolutional layer is applied a Max-Pooling, which is a down-sampling technique that helps reduce the spatial dimensions of the feature maps produced by the convolutional layer while retaining the most important information. It is particularly useful for controlling the size of the model and increasing its translation invariance. Finally, to end the convolutional block we apply drop-out to prevent overfitting and improve the generalization ability of the model.

The fully connected blocks at the end of the convolutional neural networks take the spatially organized features extracted by preceding convolutional and pooling layers and transform them into a format suitable for making predictions.

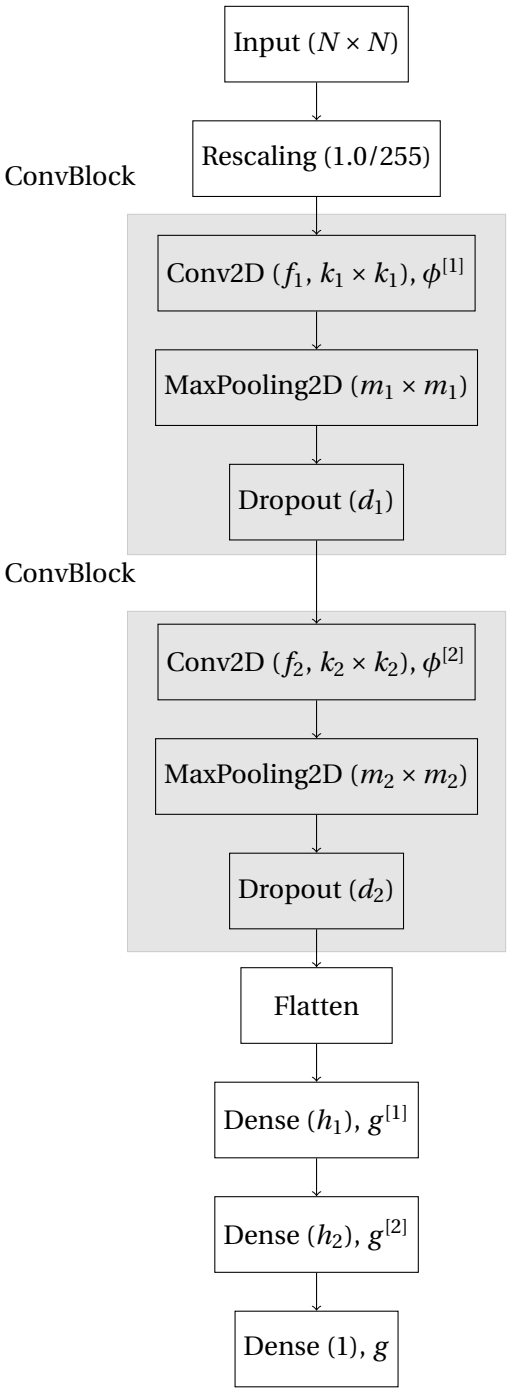

Figure 4: CNN

Parameters configuration

Weights and bias initialization, and loss functions are identical to the previous models. The fully connected layers have $h_i = 128$ units both of them, identical to the multilayer perceptron described before. The activation functions of the fully connected layers $g^{[i]}$ are both ReLU functions, and the output layer activation functions g is sigmoid as previous models.

The convolutional blocks have both same parameters' configuration. Starting from the filters, both have $f_i = 64$ filters and a kernel size of $k_i = 3$. Additionally, the activation functions $\phi^{[i]}$ are both ReLU as well. The max-pooling has same size of $m_i = 2$ in both of them. Finally, the dropout rate is $d_i = 0.25$ in both blocks.

3 Results

The results are based on the analysis of the *anuka* dataset which contains 1200 instances of each pollen grain class: kunzea and lepto. The data is randomly split into stratified training and validation datasets with a proportion of 0.8 for training and 0.2 for validation. The random seed used to split the data is kept constant to have always the same datasets distributions. In our case, we also applied data augmentation to the training set in order to obtain more data and improve our models' performance. The augmentation must be in such a way that the data contains the same information, therefore we applied only random flip (horizontal) and random rotation of $\theta = \pi/10$.

The evaluation metrics is the accuracy, a typical metric in classification problem for model assessment

$$J = \frac{1}{N}\sum_{i=1}^{N} \delta(L(\hat{y}_i, y_i)) \tag{4}$$

where δ is the Kronecker's delta function and L is the 0-1 loss function such that

$$L(\hat{y}_i, y) = \begin{cases} 0 & \text{if} \quad \hat{y}_i = y_i \\ 1 & \text{if} \quad \hat{y}_i \neq y_i \end{cases} \qquad \text{and} \qquad \delta(L(\hat{y}_i, y)) = \begin{cases} 1 & \text{if} \quad \hat{y}_i = y_i \\ 0 & \text{if} \quad \hat{y}_i \neq y_i \end{cases} \tag{5}$$

We conducted batch training with a batch size of 128 and the number of epochs fixed at 500 for all the different models. Furthermore, we employed the Adam optimizer for training our models with an initial learning rate of 0.01. The training for logistic regression and multilayer perceptron was done on my personal computer using CPU, nevertheless to increase the computations resources I used Google Colaboratory where there are GPUs available for free use. The software used to construct the different neural networks was Keras with TensorFlow backend for Python 3.10.

3.1 Logistic Regression

The training progress of the logistic regression model is showed in Figure 5. The loss function, namely binary cross-entropy (2), along different epochs, appears in the left panel. On the other hand, the evaluation metric, that is, the accuracy appears in the right panel. The training accuracy reach a level of around $J_T \sim 0.83$ at a lower epoch and then keeps its value mostly constant, while the validation accuracy is around $J_V \sim 0.78$. It is worth noting that the training set error never reach zero, and it seems that the model is saturated, meaning that more training epochs does not imply lower training error. Indeed, the progress does not show a tendency that clearly indicates if the model is learning. Finally, there is no overfitting as you can easily check.

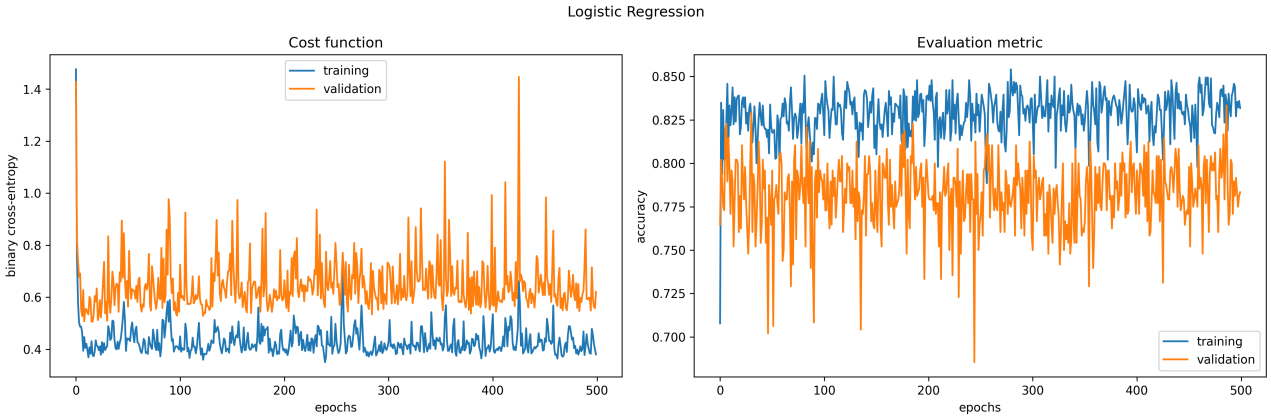

Figure 5: Logistic Regression training and validation

.

3.2 Multilayer Perceptron

The training process of the multilayer perceptron is showed in Figure 3. The training error reach almost zero after the training. The validation accuracy is 0.90 after the training. The left panel of Figure 3 shows how the training loss function is decreasing over epochs while the validation is increasing, which could results in an overfitting to the training data.

3.3 Convolutional Neural Network

The training process of the convolutional neural network is showed in Figure 4. The training error does not reach zero, but it is close to it at 0.01. The validation accuracy is 0.94 after the training. The left panel of Figure 4 shows how the training loss function is decreasing over epochs while the validation is increasing, which could results in an overfitting to the training data.

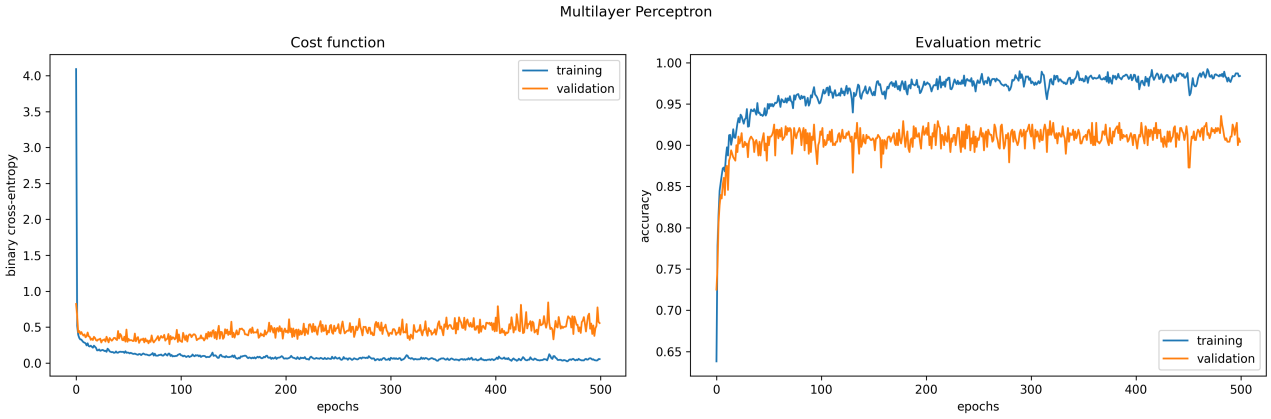

Figure 6: Multilayer perceptron training and validation

.

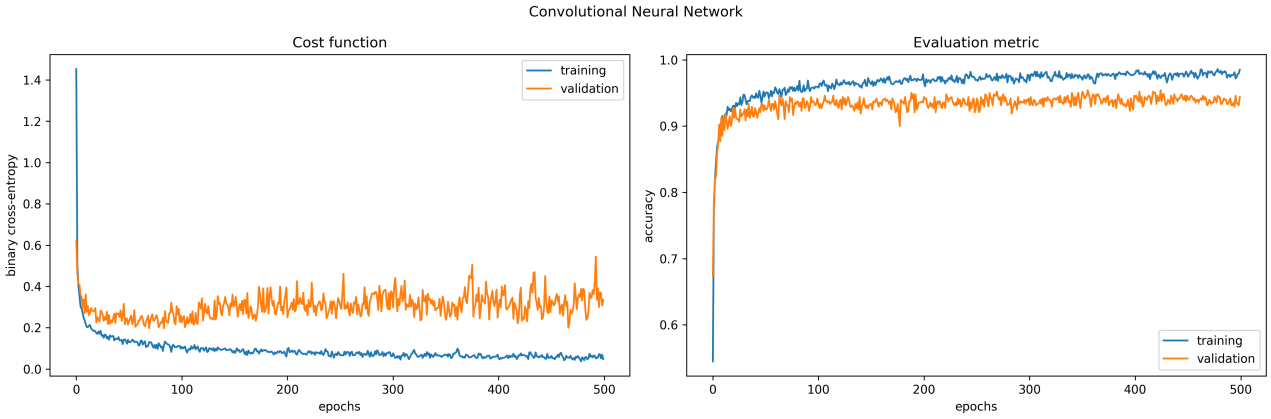

Figure 7: Convolutional Neural Network training and validation

.

4 Discussion

The different neural networks implementation offers different results. The worst of them comes from the logistic regression. The multilayer perceptron improves this performance, but fails to generalize showing lower test accuracy. Finally, the CNN offers the best results up to 4 % more accurate than MLP and 6 % more than LR as you can see in Table 1. The progress of test set performance of the models is shown in Figure testcurv

Network Architecture	Links	Weights	% Correct
Logistic Regression	30001	30001	78.3 %
Multilayer Perceptron	3856769	3856769	90.4 %
Covolutional Neural Network	4389057	4389057	94.3 %

Table 1: Test set performance of three different neural networks.

5 Supplementary models

I have tested other CNN topologies in order to find a trade-off between complexity and performance. I have trained a simpler CNN model with the same parameters but only one convolutional block, given the same results as a simpler MLP, therefore I increased the complexity with two convolutional blocks as showed previously. I also checked the same structure with three convolutional blocks and the results are showed in Figure 9.

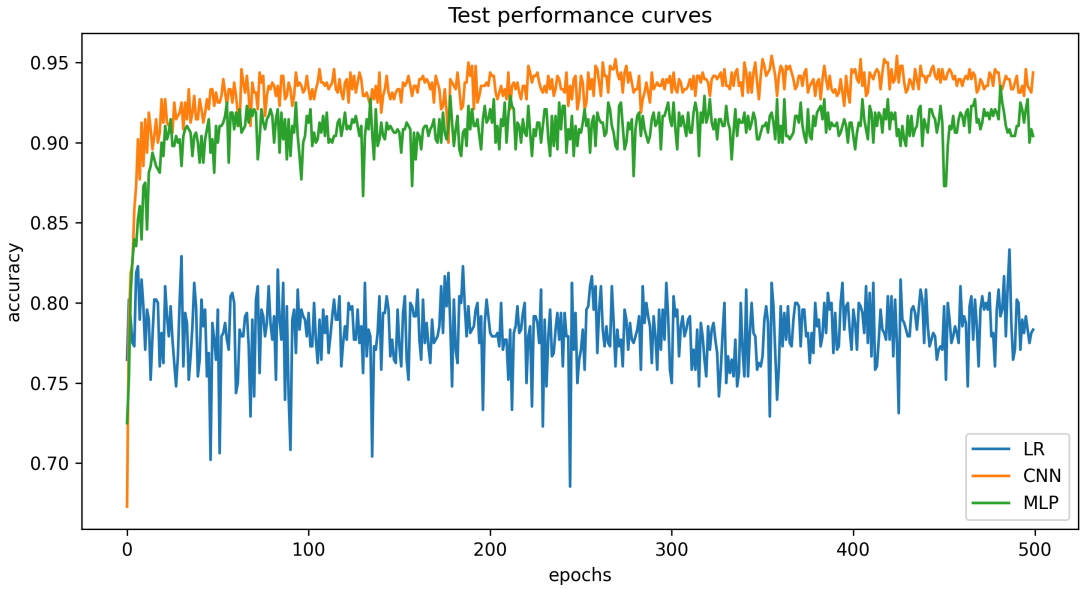

Figure 8: Logistic Regression training and validation

.

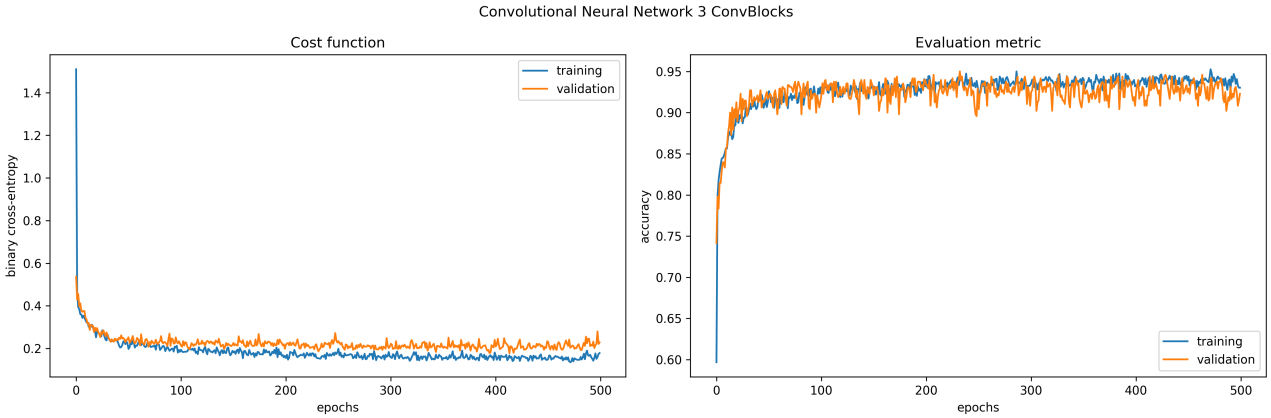

Figure 9: Alternative Convolutional Neural Network training and validation

.

Finally, I also wanted to check the performance of more complex models. Thus, I took one from internet, but even the training error was zero, the generalization performance was bad. The model was fitting the noise. The results were that bad that I decided not to include them here.